```
a="Learning Python is easy"
l=a.split(" ")
print(l[::-1])
```

```
['easy', 'is', 'Python', 'Learning']
```

```
for i in l:
  print(l[4-i])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_7452/38187824.py in <cell line: 0>()
      1 for i in l:
----> 2   print(l[4-i])

TypeError: unsupported operand type(s) for -: 'int' and 'str'
```

```
b="Apple"
l1=[]
for i in range(5):
  l1.append(b[4-i])
```

```
print(l1)
```

```
['e', 'l', 'p', 'p', 'A']
```

```
list1=[1,2,3,5,4,3,2,1,5,2]
for i in list1:
  for j in list1:
    if(list1[i]==list1[j]):
      del(list1[j])

print(list1)
```

```
[1, 2, 3, 5, 4, 3, 2, 1, 5, 2]
```

```
v=123 # reverse this number
b=0
b=(v%100)+b
v=v-(v%100)
print(v,b)
```

```
100 23
```

```
# OOP: class: logical representation of things, object: physical representation of things (class)??
# instance memory?? [RAM->Heap memory->instance memory]; class/method area???
```

```
class Fruit:
  def show(self):
    print("Apple is a fruit")

obj=Fruit() # Allocated object
obj.show()

Fruit().show() # Anonymous object
```

```
Apple is a fruit
Apple is a fruit
```

```
# Constructor: is used to initialise object's state
# Purpose: It is used to initialise member variable of invoking object
# Syntax: __init__(self)
# Destructor ka syntax: __del__(self)

class Fruit:
  def __init__(self):
    print("Fruits are ambiguous??")
  def show(self):
    print("Apple is a fruit")

obj=Fruit()
obj.show()
```

```
Fruits are ambiguous??
Apple is a fruit
```

```
# ?? __a=18; here a is a private variable
class Objects:
  def __init__(self):
    self.a="Too much coffee is dangerous for your health!" # variable using self keyword is an instance variable
  def show(self):
    print(self.a)
    print("But how much exactly is safe?")
    print("I don't know about that, but half a litre is definitely not.")

obj=Objects()
obj.show()
```

```
Too much coffee is dangerous for your health!
But how much exactly is safe?
I don't know about that, but half a litre is definitely not.
```

```
class abc:
  def __init__(self):
    self.Name="Atharva"
    Class= 8
    Marks= 100
  def show(self):
```

```
    print("Name: ", self.Name)
    print("Further data access denied")

obj=abc()
obj.show()
```

```
Name:  Atharva
Further data access denied
```

```
# class with parameters
class abc:
  def __init__(self,p,q,r):
    self.Name=p
    self.Class=q
    self.Marks=r
  def show(self):
    print("Name: ", self.Name)
    print("Class: ", self.Class)
    print("Marks: ", self.Marks)


obj=abc("Anit",'...',98)
obj1=abc("Atharva",'...',100)
obj.show()
obj1.show()
```

```
Name:  Anit
Class:  ...
Marks:  98
Name:  Atharva
Class:  ...
Marks:  100
```

```
class abc:
  def __init__(self,p,q,r):
    self.Name=p
    self.Age=q
    self.Marks=r
  def show(self):
    print("Name: ", self.Name)
    print("Age: ", self.Age)
    print("Marks: ", self.Marks,"\n")

a=input("Enter student name:")
b=int(input("Enter student age: "))
c=float(input("Enter student's marks: "))
obj=abc(a,b,c)
a=input("Enter student name:")
b=int(input("Enter student age: "))
c=float(input("Enter student's marks: "))
obj1=abc(a,b,c)
```

```
obj.show()
obj1.show()
```

```
Enter student name:Atharva
Enter student age: 20
Enter student's marks: 99
Enter student name:Vaibhavi
Enter student age: 20
Enter student's marks: 99
Name:  Atharva
Age:  20
Marks:  99.0

Name:  Vaibhavi
Age:  20
Marks:  99.0
```

```
class pqr:
  def  __init__(self):
    print("Andromeda is going to collide with Milky Way")
  def __init__(self):
    print("It's a secret")

obj=pqr() # constructor and function overloading does not exist in Python, last cinstructor will be considered
```

```
It's a secret
```

```
# Deleting instance variable
class Hair:
  def __init__(self):
    self.a="Straight"
    self.b="Curly"
    self.c="No hair (bald)"
    self.d="Wavy"
  def fucn(self):
    del self.d

  s=Hair()
  print(s.__dict__)
  s.func()
  print(s.__dict__)
  del s.a
  print(s.__dict__)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_7452/3107779250.py in <cell line: 0>()
      1 # Deleting instance variable
----> 2 class Hair:
      3   def __init__(self):
      4     self.a="Straight"
      5     self.b="Curly"

/tmp/ipykernel_7452/3107779250.py in Hair()
      9     del self.d
     10
---> 11   s=Hair()
     12   print(s.__dict__)
     13   s.func()

NameError: name 'Hair' is not defined
```

```
# Destructor
class abc:
  def __init__(self):
    print("Program started")
    self.Name="Atharva"
    # Class= 8
    # Marks= 100

  def __del__(self):
    print("Destructor has deleted the obj")

  def show(self):
    print("Name: ", self.Name)
    print("Further data access denied")

obj=abc()
obj.show()
```

```
Program started
Destructor has deleted the obj
Name:  Atharva
Further data access denied
```

```
class happy:
  x=90
  def __init__(self):
    self.y=10

o1=happy()
o2=happy()
print("For o1:", o1.x,o1.y)
print("For o2:", o2.x,o2.y)
happy.x=100
o1.y=20
```

```
print("For o1:", o1.x,o1.y)
print("For o2:", o2.x,o2.y)
```

```
For o1: 90 10
For o2: 90 10
For o1: 100 20
For o2: 100 10
```

```
class argument:
  def a1(self):
    a=100
    print(a)
  def a2(self):
    b=100
    a=200
    print("\n",a,"\n",b)

obj=argument()
obj.a1()
obj.a2()
```

```
100

 200
 100
```

```
# Accept 5 paper marks, name of student, rollno., calc total and percentage, check if %>=85, then student can take admission, else, cannot take admission
class Student:
  m=[]
  sum=0
  def __init__(self):
    print("Enter student details:\n")
    n=input("Name:")
    r=int(input("Roll no.:"))
    for i in range(5):
      m[i]=float(input("Marks:"))

  def tncheck(self):
    for i in range(5):
      sum=sum+m[i]

    percentage=sum/5
    print("Total=",sum)
    print("Percentage=",percentage)
    if(percentage<85):
      print("Cannot admit")
    else:
      print("Can admit")

obj=Student()
obj.tncheck()
```

```
Enter student details:

Name:Atharva
Roll no.:12
Marks:90
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_7452/366602804.py in <cell line: 0>()
     22       print("Can admit")
     23
---> 24 obj=Student()
     25 obj.tncheck()

/tmp/ipykernel_7452/366602804.py in __init__(self)
      8     r=int(input("Roll no.:"))
      9     for i in range(5):
---> 10       m[i]=float(input("Marks:"))
     11
     12   def tncheck(self):

NameError: name 'm' is not defined
```

```
class Student:
  m=[]
  sum=0
  def __init__(self):
    print("Enter student details:\n")
    self.n=input("Name:")
    self.r=int(input("Roll no.:"))
    self.m1=float(input("Marks1:"))
    m2=float(input("Marks2:"))
    m3=float(input("Marks3:"))
    m4=float(input("Marks4:"))
    m5=float(input("Marks5:"))
  def tncheck(self):
    sum=m1+m2+m3+m4+m5

    percentage=sum/5
    print("Total=",sum)
    print("Percentage=",percentage)
    if(percentage<85):
      print("Cannot admit")
    else:
      print("Can admit")

obj=Student()
obj.tncheck()
```

```
#  Inheritance: single, multiple, multilevel, heirarchical, hybrid
class A:
  def showa(self):
```

```
    print("First func")
class B(A):
  def showb(self):
    print("Second func") # single inheritance

obj=B()
obj.showa()
obj.showb()
```

```
First func
Second func
```

```
class A:
  def showa(self):
    print("First func")
class B(A):
  def showb(self):
    print("Second func") # multilevel inheritance

class C(B):
  def showc(self):
    print("Third func")

obj=C()
obj.showa()
obj.showb()
obj.showc()
```

```
First func
Second func
Third func
```

```
class A:
  def showa(self):
    print("First func")
class B:
  def showb(self):
    print("Second func") # multiple inheritance

class C(A,B):
  def showc(self):
    print("Third func")

obj=C()
obj.showa()
obj.showb()
obj.showc()
```

```
First func
Second func
Third func
```

```
class A:
  def showa(self):
    print("First func")
class B(A):
  def showb(self):
    print("Second func") # heirarchical inheritance

class C(A):
  def showc(self):
    print("Third func")

obj=B()
obj.showa()
obj.showb()
obj1=C()
obj1.showa()
obj1.showc()
```

```
First func
Second func
First func
Third func
```

```
class A:
  def showa(self):
    print("First func")
class B(A):
  def showb(self):
    print("Second func") # hybrid inheritance

class C(A):
  def showc(self):
    print("Third func")

class D(B,C):
  def showd(self):
    print("Fourth func")

obj=D()
obj.showa()
obj.showb()
obj.showc()
obj.showd()
print("\n")
obj1=B()
obj.showa()
obj.showb()
print("\n")
obj2=C()
obj.showa()
obj.showc()
```

```
First func
Second func
Third func
Fourth func


First func
Second func


First func
Third func
```

```
# Polymorphism
# Implementation: overloading (assigning additional task)  and overriding (func name is same different classes, and both classes should be in inheritance)
class SS:
  def planet(self):
    print("Jupiter")
class Uni(SS):
  def planet(self):
    print("Pluto")

obj=Uni()
obj.planet() # this is func overriding solar system is being overriden by the universe
```

```
Pluto
```